Lecture 6: ML Best Practices & MLOps

Alexis BOGROFF

Course on Unsupervised Learning Problems @Albert School

April 8, 2026

Roadmap

ALBERTSCHOOL

(1) The Environment: Development vs. Production

(2) Experiment Tracking with MLflow

(3) Model Registry & Lifecycle

(4) Monitoring: Data Drift & Concept Drift

(5) Statistical Tests for Drift Detection (Evidently)

The "Notebook Trap"

ALBERTSCHOOL

The reality of Data Science:

- Jupyter Notebooks are fantastic for Exploratory Data Analysis (EDA), quick prototyping, and visualization.
- **But Notebooks are NOT Production.**
- They encourage bad practices: out-of-order execution, hidden states, dead code, and difficult version control (Git conflicts with JSON metadata).

The Goal of MLOps:

- Transitioning a model from a "research artifact" on your local machine to a "reliable software component" that creates continuous business value.

Development vs. Production Environments

ALBERTSCHOOL

Aspect	Development (Research)	Production (Deployment)
Format	Jupyter Notebooks (`.ipynb`)	Python Scripts (`.py`), Modules, APIs
Goal	Discover patterns, try algorithms	Serve predictions reliably and quickly
Data	Static datasets (CSV, local SQL)	Live streams, Data Warehouses, APIs
Execution	Manual (Shift+Enter)	Automated (Airflow, CRON, CI/CD)
Focus	Accuracy, Inertia, Silhouette	Latency, Uptime, Memory usage

Anatomy of a Production ML Pipeline

ALBERTSCHOOL

The 6 stages that separate a notebook from a product:

1. **Data Ingestion:** Pull live data from a data warehouse, API, or Kafka stream — not a static CSV.
2. **Feature Engineering:** Apply the *same* preprocessing pipeline as training (StandardScaler fitted on reference data, serialized with `joblib`).
3. **Model Inference:** The trained model artifact, called by an API endpoint (`FastAPI`, `Flask`) or a batch job.
4. **Logging:** Every prediction is stored: timestamp, input features, output cluster ID, latency.
5. **Monitoring:** Statistical tests continuously compare the current feature distribution vs. the reference dataset.
6. **Retraining Loop:** When drift exceeds a threshold, a pipeline trigger fires — a new run is logged in MLflow, evaluated, and registered.

Key Principle

Every stage is a handoff. Each handoff is a potential failure point.

Code Quality: Notebook vs. Modular Script

ALBERTSCHOOL

Notebook (Research)

```
# Cell 3 (run FIRST!)
df = pd.read_csv(
    "data_v3.csv")
k = 5  # k=3, 4, 6 tried
model = KMeans(k)
model.fit(df)
```

Module (Production)

```
# src/train.py
def train(path, k):
    df = load_data(path)
    df = preprocess(df)
    model = KMeans(
        n_clusters=k,
        random_state=42)
    model.fit(df)
    return model
```

The modular version is **testable**, **version-controlled**, and **callable by any orchestrator** (Airflow, CI/CD, CRON).

Reproducibility & Containerization

Problem: *"It works on my machine"* — Python 3.9 vs. 3.11, scikit-learn 1.0 vs. 1.4.
Three levels of reproducibility:

1. **Code** — `requirements.txt` with pinned versions (`scikit-learn==1.4.0`).
2. **Environment** — `conda env`: isolates the Python version and all dependencies.
3. **System — Docker**: packages the entire OS + Python + libraries into one portable image. MLflow logs `conda.yaml` automatically.

Minimal Dockerfile

```
FROM python:3.11-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY src/ src/
CMD ["python", "src/train.py"]
```

Roadmap

ALBERTSCHOOL

(1) The Environment: Development vs. Production

(2) Experiment Tracking with MLflow

(3) Model Registry & Lifecycle

(4) Monitoring: Data Drift & Concept Drift

(5) Statistical Tests for Drift Detection (Evidently)

The Need for Experiment Tracking

ALBERTSCHOOL

The typical Data Scientist problem:

"I ran 50 versions of K-Means yesterday. The 32nd one was the best. But wait... did I use $k = 5$ or $k = 6$? And did I apply StandardScaler or MinMaxScaler? I forgot to save the plot..."

The Solution: MLflow Tracking

- An open-source platform to systematically record and query your machine learning experiments.
- It acts as a "flight data recorder" for your code.

MLflow Core Concepts

ALBERTSCHOOL

1. Tracking Server: A centralized database (often hosted on AWS/GCP) where your team's experiments are aggregated.

2. Experiment: A logical grouping of runs. (e.g., `Customer_Segmentation_Q3`).

3. Run: A single execution of your training code. Within a Run, MLflow logs three distinct elements:

- **Parameters:** The inputs (e.g., algorithm=`DBSCAN`, epsilon=0.5, min_samples=5).
- **Metrics:** The outputs/evaluations (e.g., Silhouette Score=0.72, WCSS=4500).
- **Artifacts:** Files produced (e.g., the saved model `.pkl`, the PCA Biplot `.png`, requirements.txt).

MLflow: Logging a K-Means Experiment

ALBERTSCHOOL

Python: Full K-Means Tracking Example

```
import mlflow, mlflow.sklearn
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("Customer_Segmentation_Q3")

with mlflow.start_run(run_name="kmeans_k5"):
    mlflow.log_param("k", 5)
    mlflow.log_param("scaler", "StandardScaler")
    X_s = StandardScaler().fit_transform(X)
    model = KMeans(n_clusters=5, random_state=42).fit(X_s)
    mlflow.log_metric("silhouette", silhouette_score(X_s, model.labels_))
    mlflow.log_metric("inertia", model.inertia_)
    mlflow.sklearn.log_model(model, "kmeans_model")
    mlflow.log_artifact("outputs/biplot.png")
```

- `with mlflow.start_run()` closes the run cleanly even if an exception occurs.
- Each `log_param()` / `log_metric()` call writes to the Tracking Server in real time.

MLflow Autolog: Power and Limits

MLflow Autolog — one line to automatically capture params, metrics, model signature, and environment:

```
mlflow.autolog()
```

Supported frameworks: `sklearn`, `xgboost`, `lightgbm`, `tensorflow`, `pytorch`.

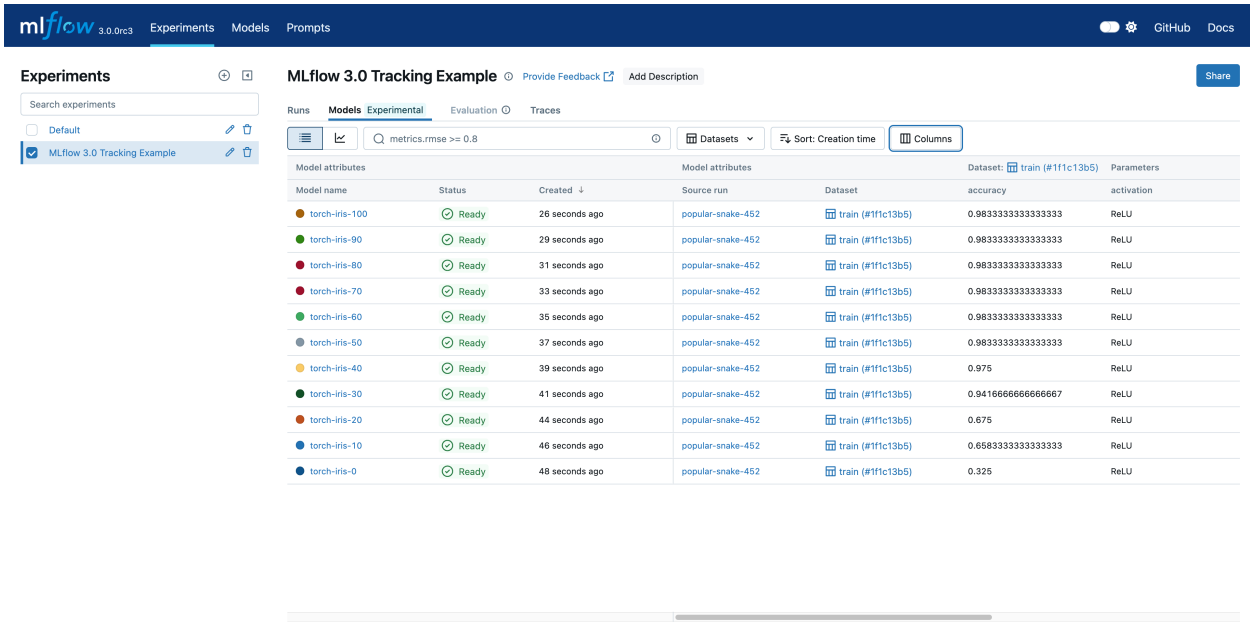
Warning for Unsupervised ML

Autolog was designed for supervised pipelines. For clustering, it **misses**:

- **No Silhouette Score** — no standard `score()` method returns a cluster quality metric.
- **No cluster membership distribution** — how many points in each cluster?
- **No centroid artifact** — centroids are critical for monitoring cluster stability over time.

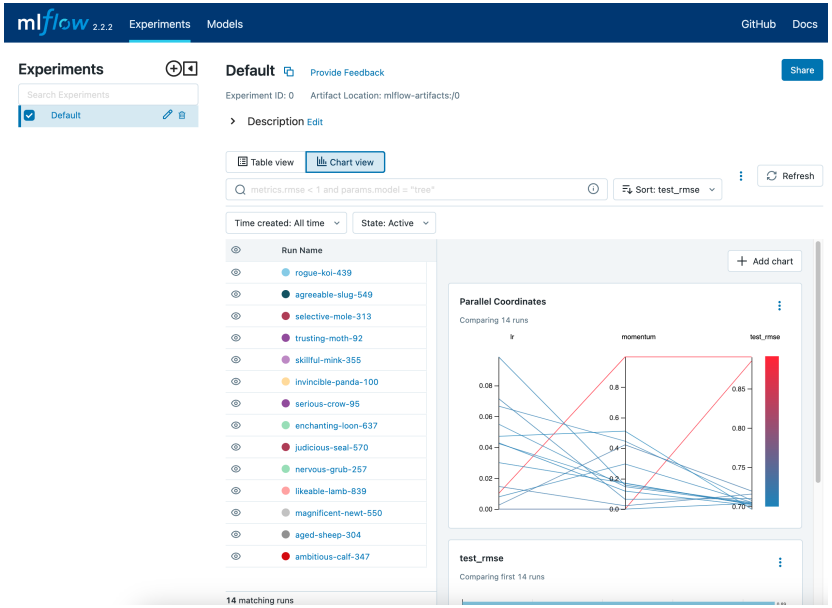
Best practice: Use autolog as a baseline; always add manual `mlflow.log_metric()` calls for your unsupervised quality metrics.

MLflow UI: Visualizing Experiments



- Instantly compare hundreds of runs.
- Filter by metric (e.g., "Show me runs where Silhouette > 0.6").
- Reproducibility: Anyone on the team can download the exact environment and weights of a specific run.

MLflow UI: Comparing Multiple Runs



- Select multiple runs and click **Compare**: instantly see parameter/metric charts side by side.
- **Parallel Coordinates Plot:** reveals which parameter combinations consistently yield high Silhouette scores.
- A team of 5 data scientists can run experiments in parallel — all land in the same Experiment, fully auditable.

From Run to Reproducible Artifact

ALBERTSCHOOL

Every run URI is a time machine. You can reload any logged model months later, on any machine:

Python: Reloading a Logged Model

```
import mlflow.sklearn

run_id = "a3f2c1b4e5d6..."   # from the MLflow UI
model_uri = f"runs:/{run_id}/kmeans_model"

loaded_model = mlflow.sklearn.load_model(model_uri)
new_labels = loaded_model.predict(X_new_scaled)
```

- The `run_id` links to the exact parameters, metrics, environment, *and* model weights used to produce those predictions.
- This is also the input to the **Model Registry**: the best run URI is what you register for governance.

Roadmap

ALBERTSCHOOL

(1) The Environment: Development vs. Production

(2) Experiment Tracking with MLflow

(3) Model Registry & Lifecycle

(4) Monitoring: Data Drift & Concept Drift

(5) Statistical Tests for Drift Detection (Evidently)

MLflow Model Registry

ALBERTSCHOOL

What happens when the model is ready?

- Tracking is for *experiments*. The **Registry** is for *governance*.
- It is a centralized model store, set of APIs, and UI to collaboratively manage the full lifecycle of an MLflow Model.

Lifecycle Stages:

- **None:** Just logged, still evaluating.
- **Staging:** Model is deployed in a pre-production environment for integration testing.
- **Production:** The model is actively serving live business traffic.
- **Archived:** The model is retired and replaced by a newer version.

Registering a Model: Concept

ALBERTSCHOOL

From Experiment to Governance

- Once the best run is identified (e.g., via the MLflow UI), it is **registered** into the central store.
- Version numbers are **auto-incremented** (Version 1, Version 2, etc.).
- Stage transitions are **auditable**: you can track exactly who moved a model from *None* $\rightarrow$ *Staging* $\rightarrow$ *Production*, and when.

No more hardcoded paths

Instead of loading a file named `model_v3_final.pkl`, your production application simply asks the Registry to provide the model that is currently tagged as `"Production"`.

Why Model Versioning Matters

ALBERTSCHOOL

The Monday Morning Scenario

"We deployed a new model on Friday. Saturday morning: revenue dropped 12%. Which version is live? Who deployed it? What changed?"

Three concrete reasons to version your models:

1. **Rollback:** If Version 4 degrades performance, roll back to Version 3 in a single click or API call — no retraining required.
2. **Audit Trail:** Regulators (especially in Finance) require proof of which model made which decision on which date. The Registry provides this.
3. **A/B Testing:** Serve Version 3 to 80% of traffic, Version 4 to 20%. Compare Silhouette drift and business KPIs over two weeks before full promotion.

The Champion / Challenger Pattern

ALBERTSCHOOL

Deploying without breaking things:

- **Champion:** The active model currently serving 100% (or the vast majority) of live business traffic.
- **Challenger:** A newly trained candidate model (e.g., trained on more recent data) that we believe could be better.

How it works in practice (A/B Testing):

- The inference service loads the Champion for 80-90% of users.
- The Challenger receives 10-20% of "shadow" or real traffic.
- We compare their performance (e.g., business metrics, stability) over a few weeks.
- If the Challenger wins, it is promoted to Champion (often via a simple alias reassignment, with **zero code change** required in production).

Roadmap

ALBERTSCHOOL

(1) The Environment: Development vs. Production

(2) Experiment Tracking with MLflow

(3) Model Registry & Lifecycle

(4) Monitoring: Data Drift & Concept Drift

(5) Statistical Tests for Drift Detection (Evidently)

Why do models fail in Production?

ALBERTSCHOOL

Models don't age like wine, they age like milk.

- A model is a frozen snapshot of the world at the time it was trained.
- But the world changes: Consumer habits evolve, inflation hits, sensors degrade, new products are launched.
- **Result:** The model's performance silently degrades over time.

The Solution: Monitoring

- We must continuously compare the **Reference Dataset** (the data used during training) against the **Current Dataset** (the live data in production).

Data Drift vs. Concept Drift

ALBERTSCHOOL

1. Data Drift (Covariate Shift)

- The distribution of the *input features* (X) changes.
- *Example:* A clustering model for bank customers was trained when the average age was 35. Three years later, a new marketing campaign attracts mostly 18-year-olds. The input features have shifted.

2. Concept Drift

- The relationship between the input (X) and the target (Y) changes.
- *Example:* In 2019, buying lots of toilet paper meant you were a restaurant owner. In March 2020, it meant you were panicking about Covid.
- **In Unsupervised Learning:** Concept drift is harder to track (no Y). We usually monitor **Cluster Stability** (are the centroids moving over time?).

Monitoring Unsupervised Models: The Special Challenge

ALBERTSCHOOL

Why is monitoring harder without labels?

- **Supervised:** Ground-truth labels eventually arrive. Compare predicted vs. actual → accuracy, F1, etc.
- **Unsupervised:** No ground truth exists. Monitoring must be **proxy-based**.

Two families of proxy metrics:

1. **Input Monitoring** — Are my feature distributions drifting?
   Detectable with statistical tests (KS, Wasserstein, PSI).
2. **Output Monitoring** — Are my cluster assignments stable?
   Requires tracking cluster structure over time (see next slide).

Key Insight

Input drift **precedes** output drift. Catching feature distribution shifts early gives you time to retrain before cluster assignments degrade.

Cluster Stability Metrics

Two concrete, measurable metrics for production unsupervised systems:

1. Centroid Displacement — How much has each cluster center moved between two scoring runs?

$$\Delta_k = \|\boldsymbol{\mu}_k^{(t)} - \boldsymbol{\mu}_k^{(t-1)}\|_2$$

where $\boldsymbol{\mu}_k^{(t)}$ is the centroid of cluster k at time t. A displacement exceeding 2σ of the historical baseline triggers a review.

2. Membership Change Rate (MCR) — What fraction of points changed cluster between two runs?

$$MCR = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left[\hat{y}_i^{(t)} \neq \hat{y}_i^{(t-1)}\right]$$

A stable model should have $MCR < 5\%$. Above 10%, investigate before retraining.

Implementation: Log centroids as a `.npy` artifact in MLflow at each periodic scoring run — this links output monitoring directly back to your experiment history.

Roadmap

ALBERTSCHOOL

(1) The Environment: Development vs. Production

(2) Experiment Tracking with MLflow

(3) Model Registry & Lifecycle

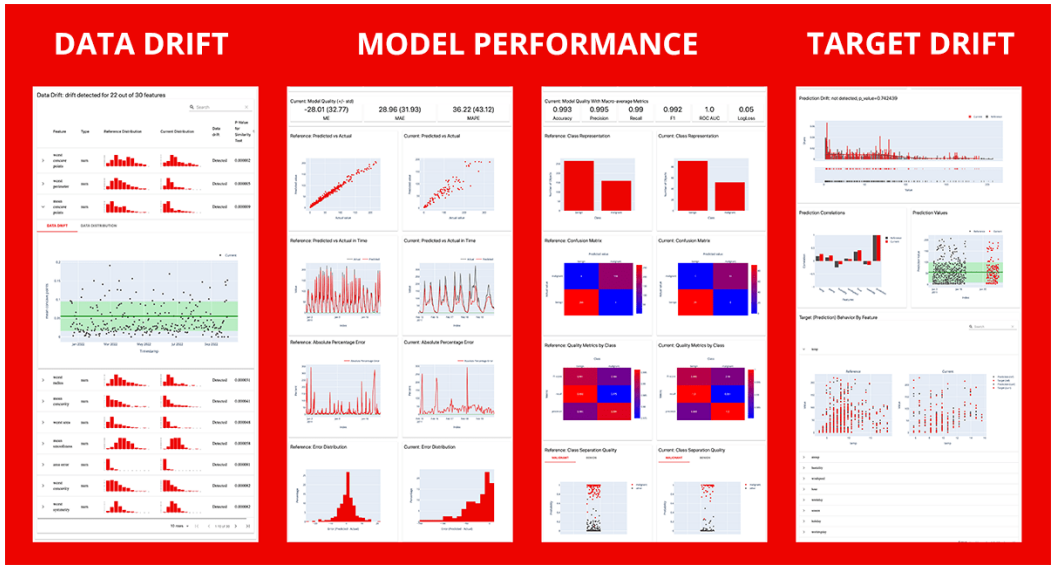
(4) Monitoring: Data Drift & Concept Drift

(5) Statistical Tests for Drift Detection (Evidently)

How to measure Drift? (Evidently AI)

We need algorithms, not just visual dashboards.

- To automate retraining pipelines, we need algorithms to tell us if a shift is *statistically significant*.
- **Evidently** is a popular open-source Python library to evaluate, test, and monitor ML models in production.



KS-Test & Wasserstein Distance (Continuous Features)

ALBERTSCHOOL

Kolmogorov-Smirnov Test (KS-Test)

- Compares the Cumulative Distribution Functions (CDFs). Finds the point of **maximum distance**:

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{curr}}(x)|$$

- Produces an exact p-value. If $p < 0.05$: drift detected.
- **Limitation:** With very large n (millions of rows), it flags **statistically significant** but **practically irrelevant** shifts. Always inspect D alongside the p-value.

Wasserstein Distance (Earth Mover's Distance)

- The minimum "cost" to transform one distribution into the other:
  $W_1(P, Q) = \int_{-\infty}^{+\infty} |F_P(x) - F_Q(x)|\, dx$
- No p-value — requires choosing a threshold manually.
- **Advantage:** Stable and interpretable even when distributions don't overlap (where KS saturates at $D = 1$). Use it to compare drift *magnitude* across features.

Chi-Squared & PSI (Categorical Features)

ALBERTSCHOOL

Chi-Squared Test (χ^2)

- Measures deviation between observed and expected frequencies:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

- Produces an exact p-value. Evidently's default for categorical columns.
- **Limitation:** Unreliable when expected counts are low (< 5 per category).

Population Stability Index (PSI)

- Works for both categorical and continuous variables (via binning):

$$PSI = \sum (\%\text{Current} - \%\text{Reference}) \times \ln\left(\frac{\%\text{Current}}{\%\text{Reference}}\right)$$

- Industry standard in Finance/Credit Scoring. Actionable thresholds:
  - $PSI < 0.1$: Stable (Green) $\quad 0.1 \leq PSI \leq 0.2$: Monitor (Yellow) $\quad PSI > 0.2$: Retrain (Red)
- **Limitation:** Sensitive to bin count choice; no formal p-value.

Which Test for Which Feature?

ALBERTSCHOOL

Feature Type	Default Test	Alternative	When to Prefer the Alternative
Continuous (age, income, centroid coords)	KS-Test	Wasserstein	KS only tells you *whether* drift exists (binary p-value). Wasserstein tells you *how much* — its value is in the same unit as the feature, so you can rank which features drifted most and by how much.
Categorical (segment ID, country code)	Chi-Squared	PSI	Chi-Squared requires at least 5 expected observations *per category* to be reliable. With high-cardinality features (many distinct values), counts per category thin out and frequently fall below this threshold. PSI handles this via binning, and its fixed thresholds (0.1 / 0.2) satisfy regulatory audit requirements in Finance.
Cluster labels ($\hat{y}$ from K-Means)	Chi-Squared on cluster size distribution	PSI + MCR & Centroid Displacement Δ_k	Chi-Squared detects that cluster *proportions* shifted, but not *why*. Adding MCR and Δ_k distinguishes between two very different causes: new data points moving between stable clusters vs. the clusters themselves deforming.

Evidently Defaults

Evidently selects the test automatically: **KS-Test** for continuous columns, **Chi-Squared** for categorical columns. You can override this to use Wasserstein or PSI if needed.

Generating a Drift Report with Evidently

ALBERTSCHOOL

Python: DataDriftPreset Report

```
import pandas as pd
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

reference_data = pd.read_csv("reference_customers.csv")
current_data   = pd.read_csv("current_customers_week42.csv")

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=reference_data, current_data=current_data)
report.save_html("drift_report_week42.html")

# Extract results programmatically for alerting
results = report.as_dict()
drift_detected = results["metrics"][0]["result"]["dataset_drift"]
if drift_detected:
    print("ALERT: Drift detected -- review retraining pipeline")
```

- *Best practice:* `mlflow.log_artifact("drift_report_week42.html")` — ties monitoring history to your experiment record.